



WIESŁAW PIETRUSZKIEWICZ

COURSE INFORMATION

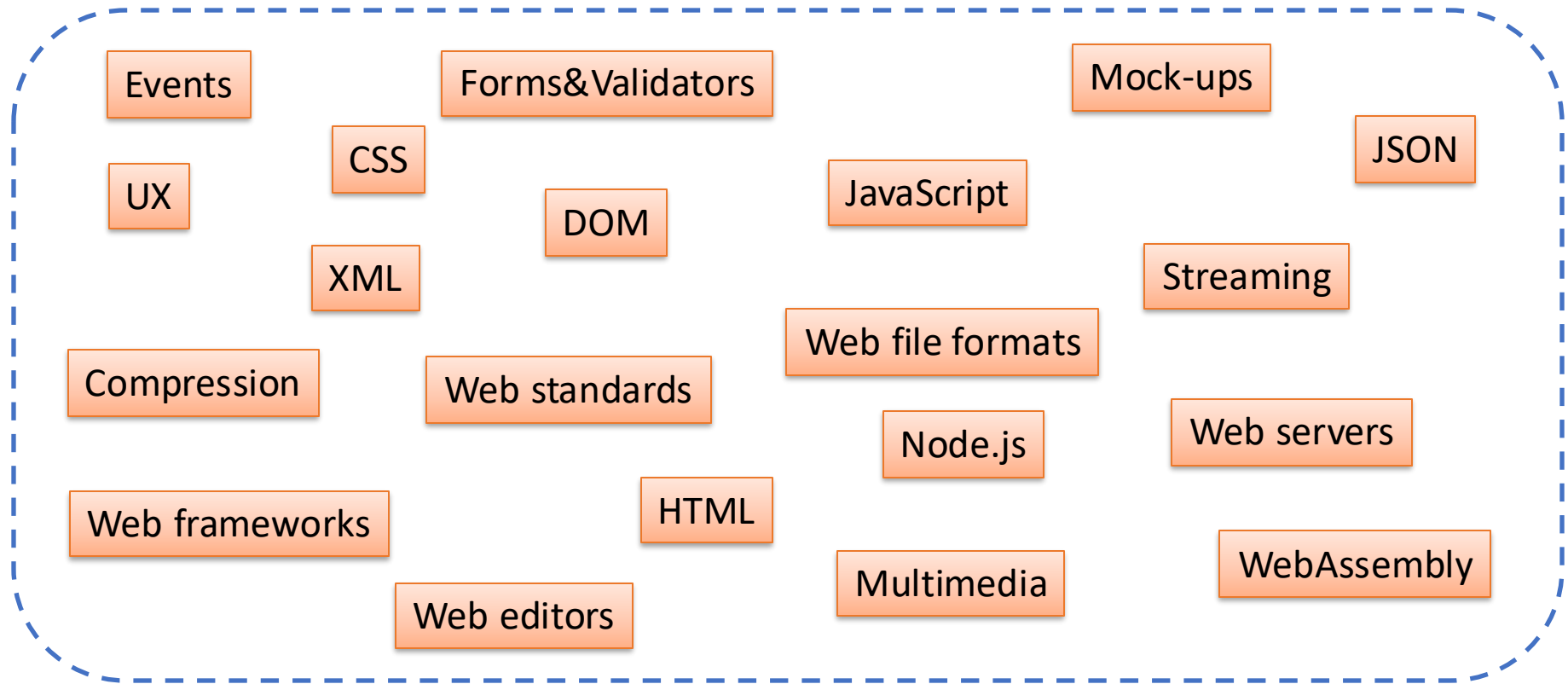
Aim

The aim of this course is to:

- Understand typical architectures of web systems
- Know technologies used to create them
- Be able to apply selected ones to create the system's front-end
- Be capable of creating development and deployment environment

Scope

We will discuss different topics including:



Evaluation

Lectures:

- Written exam
- Open-ended questions
- Based on theory and material discussed during the lectures
- Proving your proper understanding of this subject

Exercises:

- A set of individually solved exercises
- Covering different activities/technologies relating to the front-end web development

Contact

Mail: wieslaw.pietruszkiewicz@zut.edu.pl

MS Teams: Wiesław Pietruszkiewicz

Office: Building I – Room 205,

Meeting hours: Thursday 12-14

INTRODUCTION

Dynamic Documents and Front-end Web Development

Pre-Web

```
Password:
230 Logged on
Remote system type is UNIX.
ftp> ls *.c
200 Port command successful
150 Opening data channel for directory listing of "/*.c"
-rw-r--r-- 1 ftp ftp      115693 Apr 27 10:56 gc.c
-rw-r--r-- 1 ftp ftp      14289 Apr 27 10:57 gtk_functions.c
-rw-r--r-- 1 ftp ftp       902 Apr 27 10:57 map_sources.c
-rw-r--r-- 1 ftp ftp      21701 Apr 27 10:57 olc.c
-rw-r--r-- 1 ftp ftp       2993 Apr 27 10:57 os_coord_ordinance_survey.c
-rw-r--r-- 1 ftp ftp       7519 Apr 27 10:57 os_coord_transform.c
226 Successfully transferred "/*.c"
ftp> get gc.c
local: gc.c remote: gc.c
200 Port command successful
150 Opening data channel for file download from server of "/gc.c"
226 Successfully transferred "/gc.c"
115693 bytes received in 0.01 secs (17.5355 MB/s)
ftp>
```

```
Telnet csswxzpt3
User: iway
Password: ****
*****
* iWay Secure Message Broker
* Remote Administration Console
*
* locale: en_us
* iwayworkdir: C:/iWay60/config/base
* iwayconfig: base
* console-master-port: 9999
* name: Telnet_cmdline_channel
* doclocation: config
* protocol: Telnetd
* engine: base
* iway.serverip: 172.30.152.216
* iway.version: 6.0.0.SM
* iway.serverhost: csswxzpt3
* iway.serverfullhost: csswxzpt3
* iwayhome: C:/iWay60/
*
* you are logged in as iway from csswxzpt3.ibi.com (10.10.10.10)
*****
Enter command:>
```

```
Home Gopher server: gopherproject.org

THE GOPHER PROJECT
-----

Welcome to GOPHER! Gopher is a slim, powerful, and
fast way to present information in a hierarchical catalog online.
Gopher actually predates the Web -- although most web browsers
make excellent gopher browsers too.

Good places to start are the "Why Gopher?" and "Using Gopher"
areas!

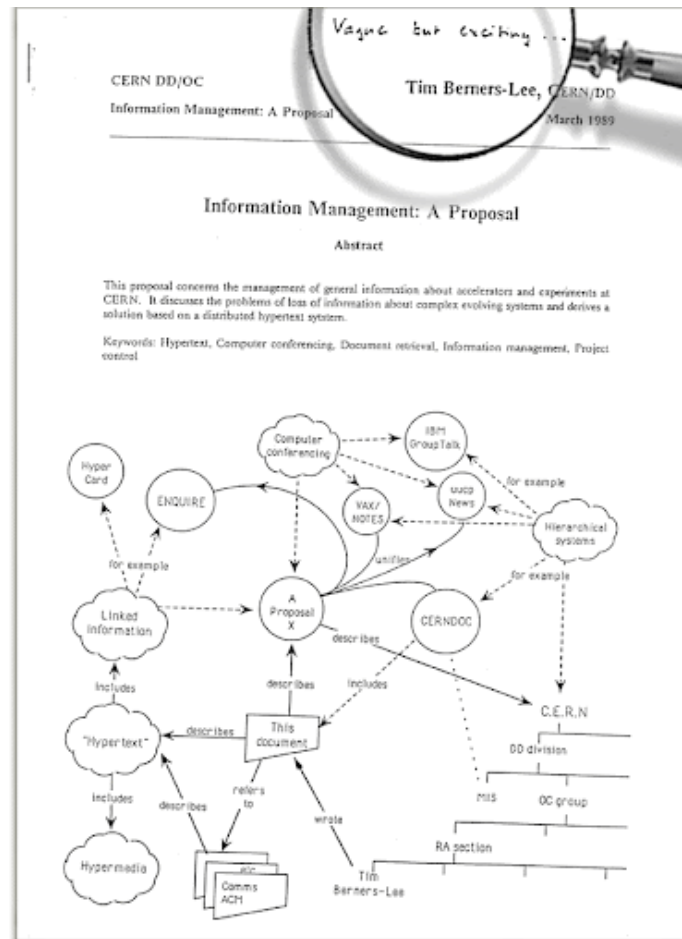
--> [12] *** GOPHER TURNS 10 / GOPHER 3.0 (FurryTerror) RELEASED ***
[13] *** GOPHER TURNS 10 ..R 3.0 (FurryTerror) RELEASED *** [html] <HTML>
[14] A Brief Introduction to Gopherspace
[15] Clients, Servers, and Downloads/
[16] Home Gopher at UMN (a good place to browse)/
[17] Home Gopher at UMN [alternate]/
[18] Mailing List
[19] Mailing List Archives/
[20] Major Gopher Servers/
[21] Screenshots/

Press h for Help, q to Quit. Page: 1/2
```

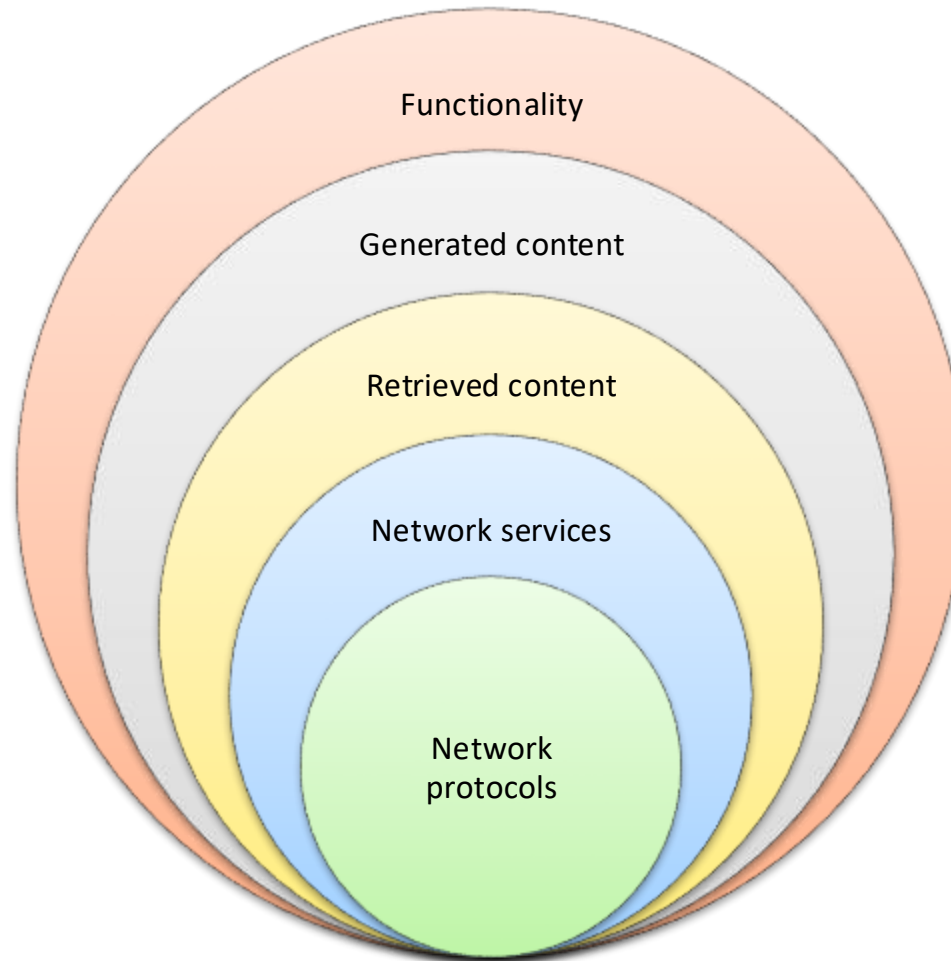
```
7:10am Josselin, Oliv. ( 23) find index of first non zero value in array
Yest. 1pm noreply, Yukih. ( 6) [ ruby-Bugs-6820 ] #ancestors never include
Yest. 2am AliasX, David,. ( 32) Bug in ruby? +ruby-talk Well, I've spent th
Nov 26 Kornelius, Yuk. ( 2) Typo in string.c +ruby-core in rb_str_ord:
Nov 26 dblack, Trans,. ( 33) [ANN] New RCRhive, including new process +
Nov 25 stef, EL, Paul. (100) Ruby vs Java vs c++ +ruby-talk hi, newbie s
Nov 22 Hugh, Eric, Ro. ( 30) Ruby 1.[89].\d+ and beyond. +ruby-core I've
Nov 20 Sean, Eric, Ji. ( 6) assert_raise question +ruby-talk Currently,
Nov 18 Brendan Ingles ( 13) YAML bug. +ruby-core Hi, First i would like
Nov 17 Soso, Yukihiro. ( 13) Accessing base method... +ruby-talk Hi all,
Nov 15 Kirill, khaine. ( 13) External entropy pool for random number gen
Nov 15 Alfonso, Morto. ( 23) ruby indentation +ruby-talk I have just st
Nov 14 David, Austin,. ( 4) Ruby on win32 cannot handle certain filename
Nov 14 Robert, Yukih. ( 3) [Pbm] Compiling ext/openssl ruby 1.8.5 on L
Nov 13 Peter, Bernard. ( 9) sprintf bug (?) +ruby-talk Hello all, I am
Nov 13 Michael, Yukih. ( 9) Ruby performance improvements +ruby-core I
Nov 12 Charles, M., D. ( 26) McGovern Likes JRuby... +ruby-talk I'm not
Nov 12 Byung-Hee, Jos. ( 32) [OT] the name of Matz +ruby-talk Hello, I s
Nov 10 Kornelius, Yuk. ( 2) What does rb_memsearch do? +ruby-core Hello
Nov 10 Ben, Trans, Ja. ( 6) accessing caller's binding implicitly +ruby
Nov 9 Wink, Yukihiro. ( 5) Threading performance +ruby-core Hello all,
Nov 9 donn, Leslie, ( 7) Kudos to all Ruby creators/enhancers/users.
[search-results-mode] search: "matz" line 12 of 22
```

Introduction to Web Technologies

And It Started With ...

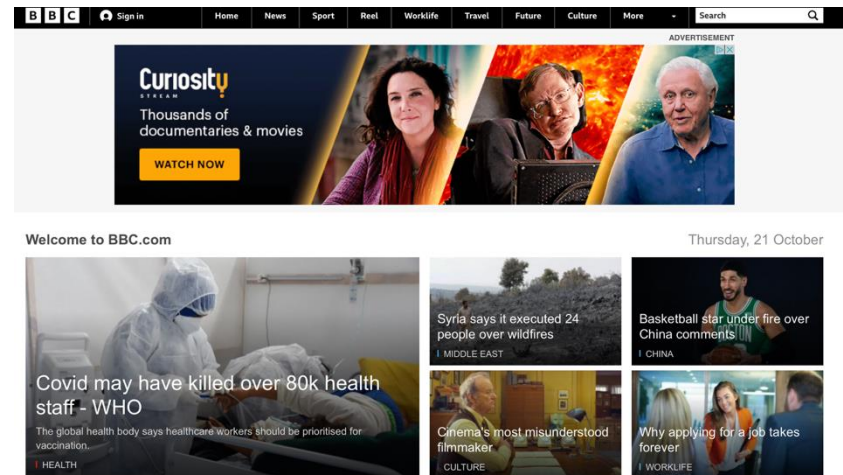


Web – Main layers



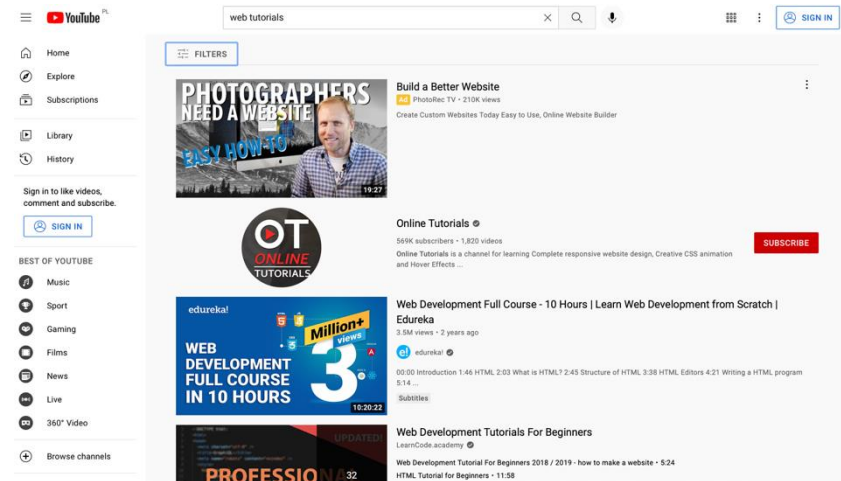
Web Sites

- Main internet service
- Focus on content
- Deliver information
- Static or dynamic
- Written from a scratch or based on templates
- Limited usage of programming libraries
- Core – HTML&JS&CSS



Web Applications

- Use more complex architecture than the one used for web sites
- Deliver functionality
- Focus on interaction
- Usually various frameworks & libraries
- Heterogenic



Mobile Web

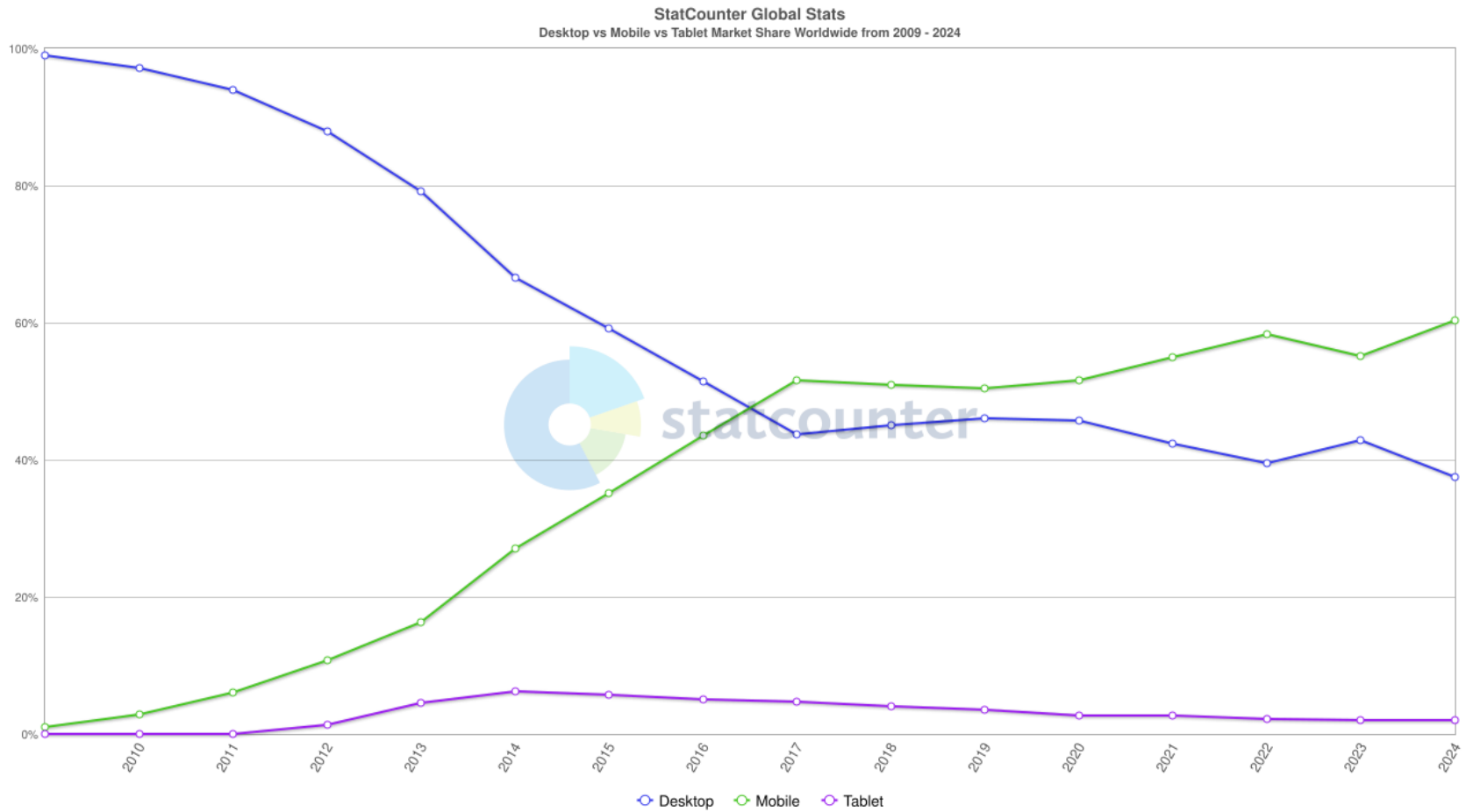
Business differences

- Web sites/apps prepared for mobile devices
- Different type of interaction
- Different usage scenarios
- A computer for many people without a “proper” computer
- Basic platform in emerging countries

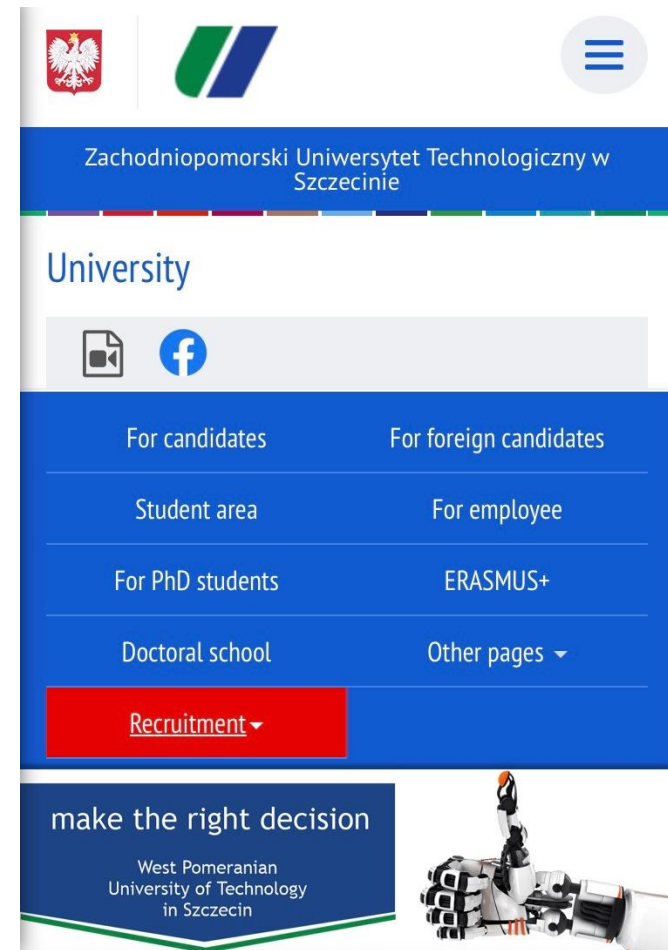
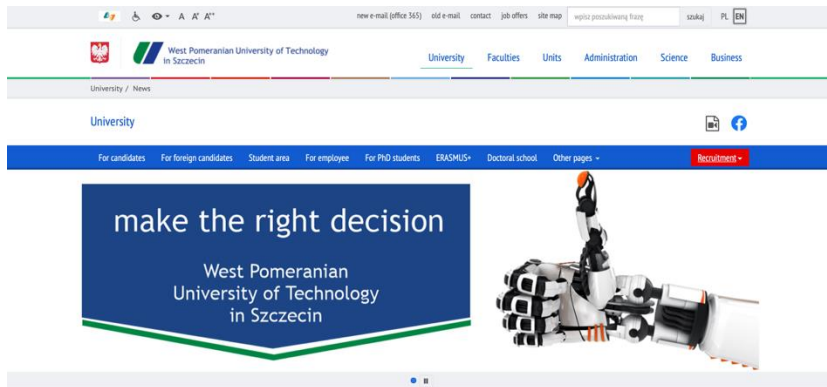
Technical differences

- Smaller display
- Lower resources
- No windows
- Difficult navigation
- Speed of rendering
- Limited addons (extensions)
- Ads are more irritating
- Limited supported standards
- Extra information (e.g. location, camera, accelerators)

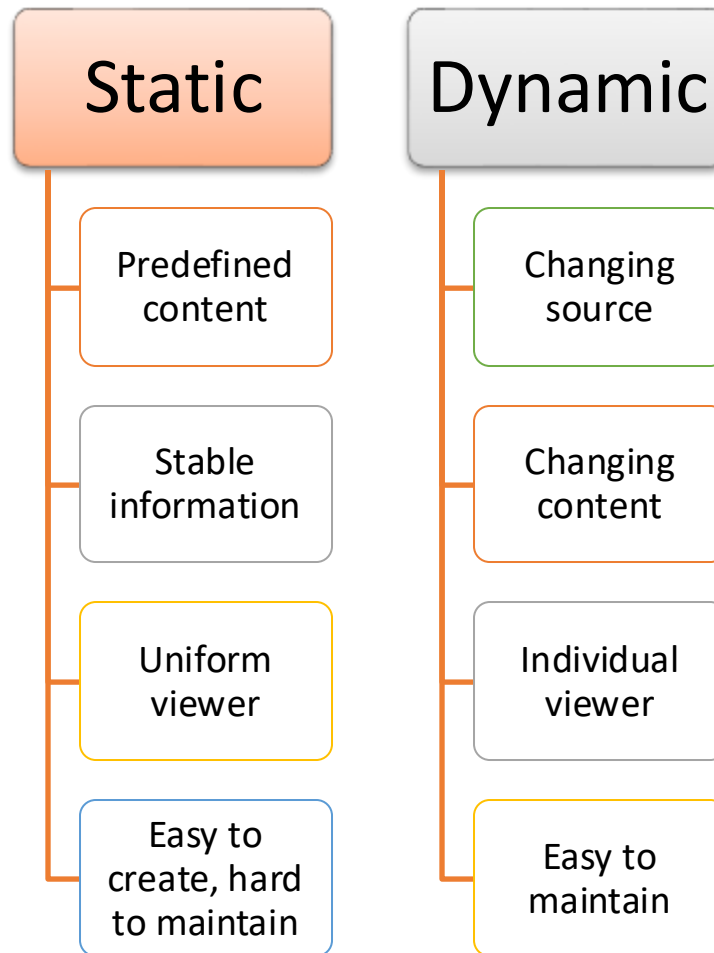
Desktop/Mobile/Tablet



Desktop vs Mobile Web Site Comparison



Static vs Dynamic Web



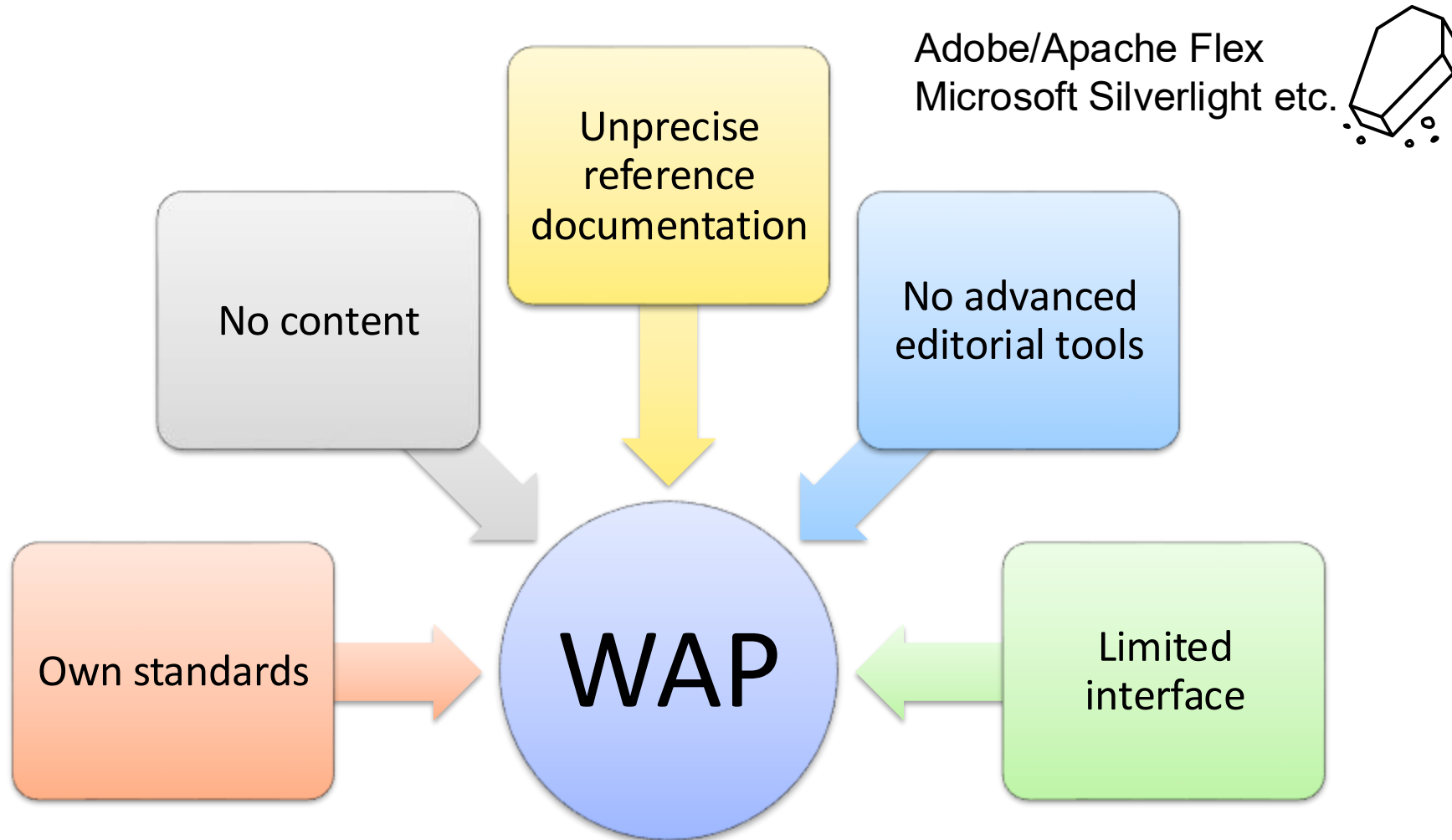
Wireless Application Protocol

One of many dead-ends

- Version 1.0 – 1999 (WML)
- Version 2.0 – 2002 (XHTML Mobile Profile)
- Version 1.0:
 - Wireless Datagram Protocol = UDP
 - Wireless Transport Layer Security = TLS
 - Wireless transaction protocol = TCP
 - Wireless Session Protocol = HTTP (compression)



R.I.P. WAP – Lesson learned



Points to remember

- Creating web solutions there are different technologies to select from
- Do not focus only on their potential
- Take into account their maturity, popularity, standardisation, offered support and experience you have
- The more novel/unique solution you deploy the bigger risk you take

Standardisation



Leading the web to its full potential

STANDARDS PARTICIPATE MEMBERSHIP ABOUT W3C

W3C » Standards

Skip

STANDARDS

W3C standards define an **Open Web Platform** for application development that has the unprecedented potential to enable developers to build rich interactive experiences, powered by vast data stores, that are available on any device. Although the boundaries of the platform continue to evolve, industry leaders speak nearly in unison about how HTML5 will be the cornerstone for this platform. But the full strength of the platform relies on many more technologies that W3C and its partners are creating, including CSS, SVG, WOFF, the Semantic Web stack, XML, and a variety of APIs.

W3C develops these technical specifications and guidelines through a process designed to maximize consensus about the content of a technical report, to ensure high technical and editorial quality, and to earn endorsement by W3C and the broader community.

If you are learning about Web technology, you may wish to start with the introduction below, and follow links for greater detail.



Web Design and Applications

Web Design and Applications involve the standards for building and Rendering Web pages, including HTML, CSS, SVG, Ajax, and other technologies for Web Applications ("WebApps"). This section also includes information on how to make pages accessible to people with disabilities (WCAG), to internationalize them, and make them work on mobile devices.



Web of Devices

W3C is focusing on technologies to enable Web access anywhere, anytime, using any device. This includes Web access from mobile phones and other mobile devices as well as use of Web technology in consumer electronics, printers, interactive television, and even automobiles.



Web Architecture

Web Architecture focuses on the foundation technologies and principles which sustain the Web, including URIs and HTTP.

SPECIFICATIONS

All Specifications

DEVELOPERS



W3C Developer Avenue is the one-stop place to find the information and tools Developers need to learn, build, and move the Web forward.



W3Cx, is where W3C, in partnership with edX, makes available Massive Open Online Courses (MOOCs) to thousands of developers worldwide.

OPEN STANDARDS PRINCIPLES



W3C

Standards

Participate

Membership

About W3C

Member Home

Introduction to Web Technologies

Typical Web Platform (Small-Mid)

LAMP

- Linux
- Apache
- MySQL/MariaDB
- PHP/Perl/Python

WAMP

- Windows
- Apache
- MySQL/MariaDB
- PHP/Perl/Python

What's the difference between Linux and Windows hosting?

← Ionos FAQ

Windows and Linux are two very different types of operating systems – each with unique advantages. Both Linux and Windows formats will work on any PC regardless of the operating system in use. However, each technology has its own distinctive characteristics. Linux web hosting is ideal if you are using PHP, MySQL or Perl programming languages, or if your website displays information in brochure, newsletter or data sheet formats. In contrast, Windows offers Windows-specific technologies such as ASP, .NET and Microsoft Access. There are also various advantages if you plan to integrate Windows-based applications or searchable databases within your website. If you aim to use out-of-the box solutions for powering content management systems or blogs, bear in mind that there are generally more open source applications available for Linux web hosting, whereas Windows licensed solutions will likely incur costs.

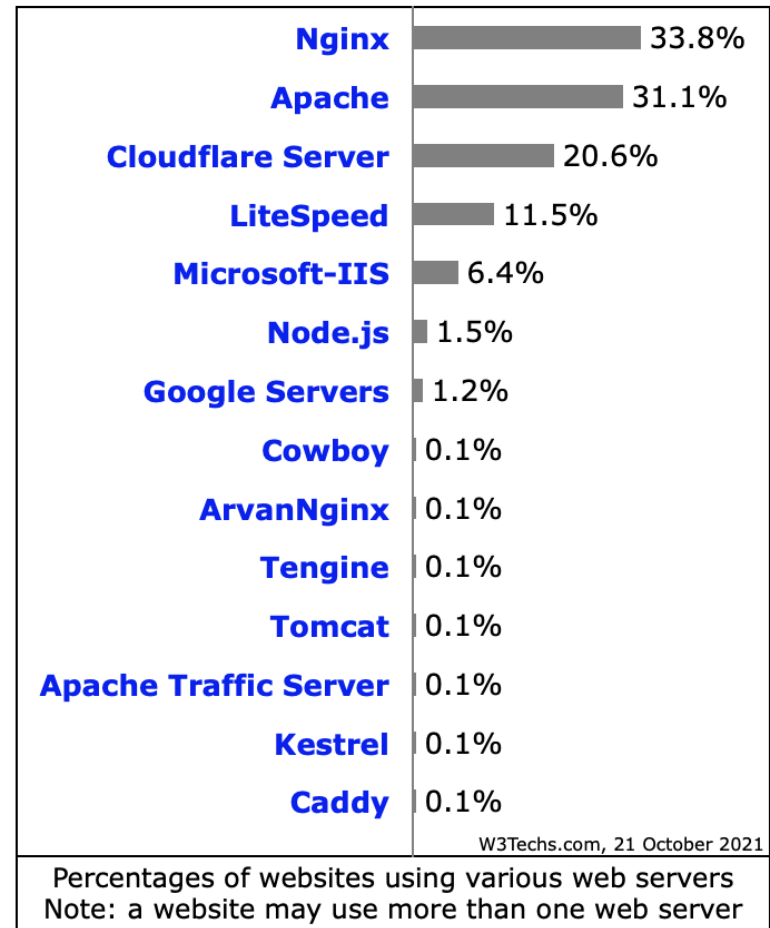
MEAN

- MongoDB
- Express.js
- Angular
- Node.js



Webserver – The Core Software

- Web sites & apps need an environment to be hosted and delivered
- Top2 – Nginx & Apache
- Server might be only a front-end server
- For some technologies choice is limited



Web solutions (sites/apps)

Advantages

- Popularity
- No installation/deployment
- Focus on user not machine
- Neutral – OS, browser
- Customisable
- Low footprint
- Information **consumption** vs production

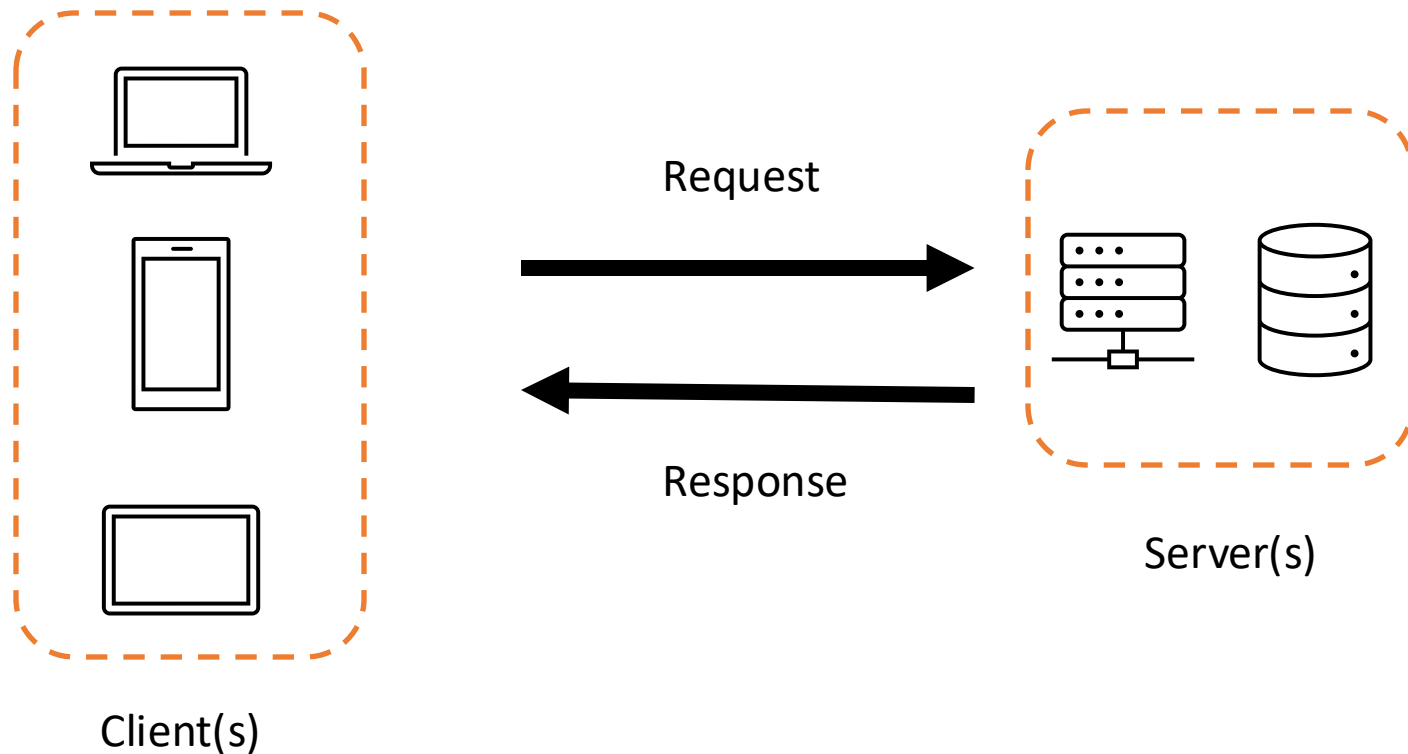
Disadvantages

- Complex stack of technologies
- Increased risk of fault
- Selected testing environments
- Transparent, if code was sent
- Lower performance
- Harder to optimise
- Information consumption vs **production**

Models/Patterns of Web Architecture

- Client-Server
- Microservices (Service-Oriented Architecture)
- Serverless
- Progressive Web Apps (PWA)
- Single-Page Applications (SPA)

Client-Server Pattern



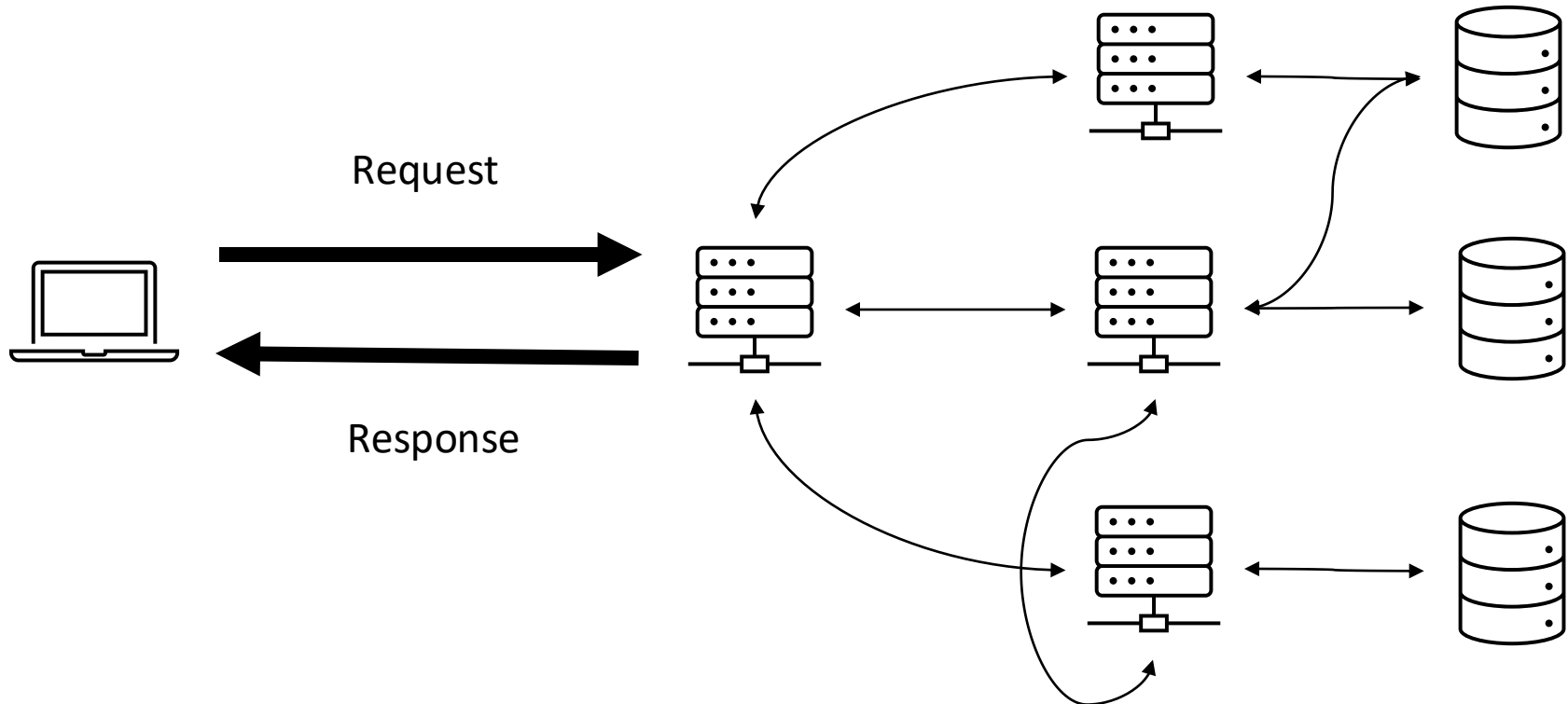
Client-Server Pattern #2

- The client-server pattern is a crucial architectural model in web systems that defines how clients and servers interact.
- The client is typically a web browser or application that interacts with the user, sending requests to the server for resources.
- The server processes these requests, retrieves or manipulates data, and sends back the appropriate responses to the client.

Client-Server Model #2

- This interaction usually occurs over a network using protocols such as HTTP (for web requests) or WebSocket (for real-time communication).
- The server acts as a centralized repository for data and application logic, which means that important information is not scattered across multiple clients, enhancing data consistency and security.
- The architecture supports scalability, allowing multiple clients to access the same server. Additional servers can be added to manage increasing traffic.

Microservices (SOA) Model



Microservices

A collection of loosely coupled services

Independent deployment

- Each microservice can be developed, deployed, and scaled independently, allowing for faster updates and feature releases.

Technology Agnosticism

- Different microservices can be built using various programming languages and technologies, suitable for their specific requirements.

Single Responsibility Principle:

- Each microservice is designed to perform a specific business function, promoting a clear focus on its purpose.

Decentralized Data Management

- Microservices can manage their own databases, enabling them to optimize data storage and usage according to their specific needs.

Enhanced Fault Isolation

- If one microservice fails, it does not necessarily impact the entire system, improving overall resilience.

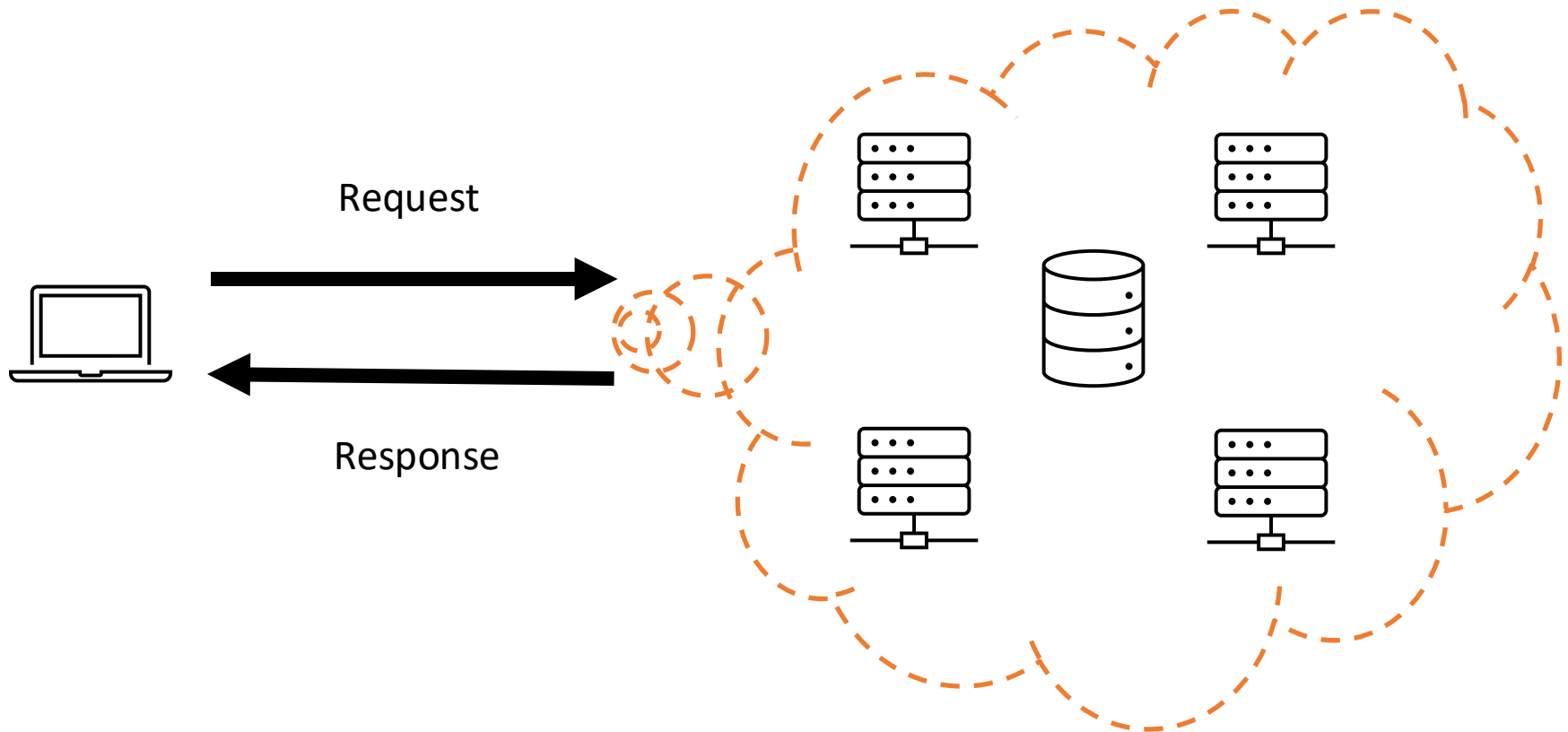
API-Driven Communication

- Microservices typically interact through APIs, allowing them to communicate seamlessly and collaborate on complex tasks.

Agility and Flexibility

- Development teams can work on different microservices simultaneously, enhancing collaboration and accelerating the overall development process.

Serverless Model



Serverless Model

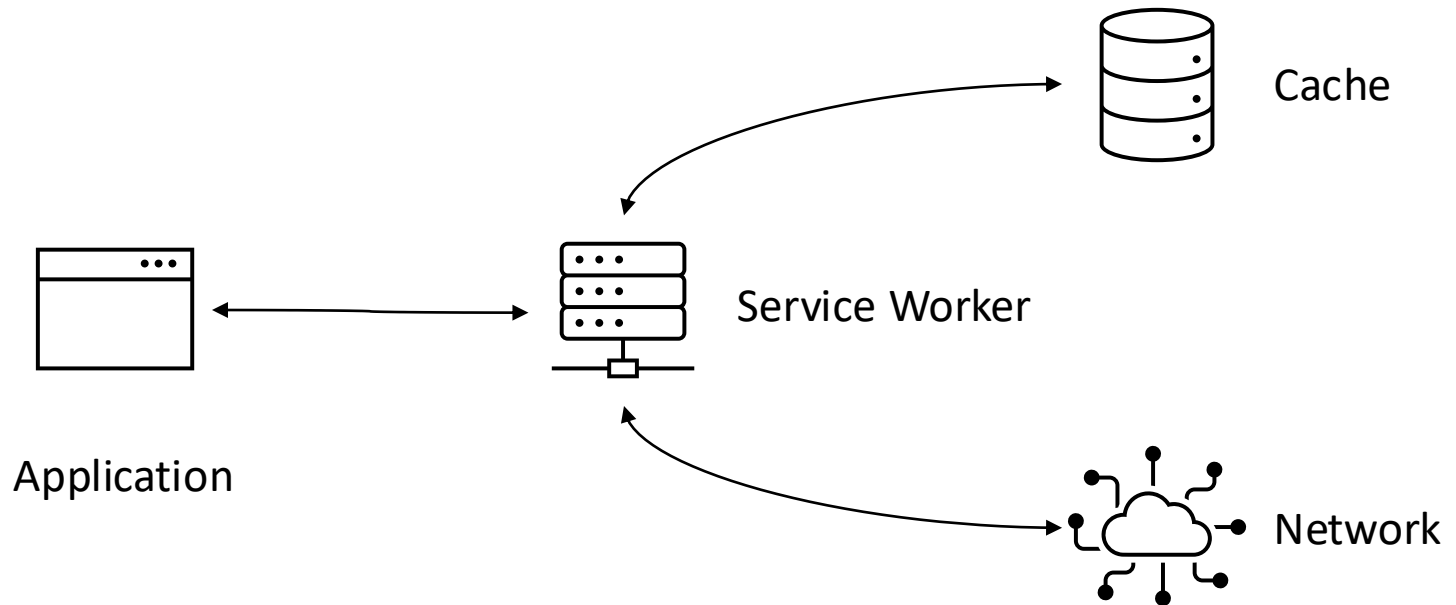
“A cloud service category where the customer can use different cloud capability types without the customer having to provision, deploy and manage either hardware or software resources, other than providing customer application code or providing customer data. Serverless computing represents a form of virtualized computing”

ISO/IEC 22123-2:2023

Information technology — Cloud computing

Part 2: Concepts

Progressive Web Apps



PWA #2

A hybrid of web and mobile applications

Offline Capabilities

- PWAs can function without an internet connection by utilizing service workers to cache resources, enabling users to access content anytime.

Responsive Design

- Designed to work seamlessly across various devices and screen sizes, providing a consistent user experience.

App-like Experience

- PWAs offer a native app feel, complete with smooth navigation and interactions, enhancing user engagement.

Installation on Home Screen

- Users can install PWAs directly on their devices' home screens without going through app stores, making access quick and easy.

Automatic Updates

- PWAs automatically update in the background, ensuring users always have the latest version without manual intervention.

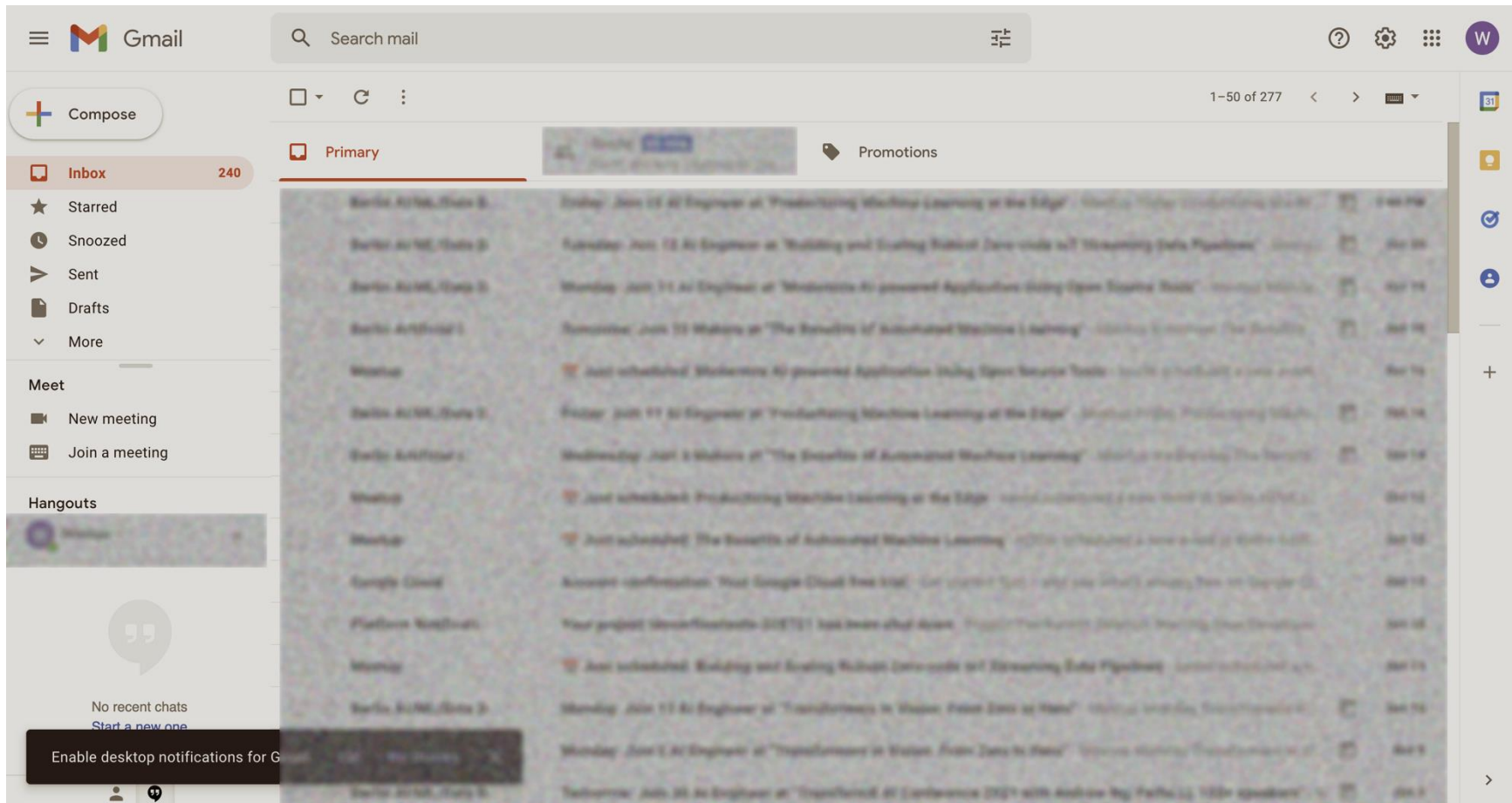
Enhanced Performance

- By leveraging caching and efficient loading techniques, PWAs provide fast and reliable performance, even on slow networks.

Secure Context

- They must be served over HTTPS, ensuring data security and trustworthiness.

Single-Page Applications



Single-Page Applications #2

Interaction without page reloading

Reduced Load Time

- SPAs load a single HTML page and dynamically update content, which minimizes server requests and improves loading speed.

Enhanced User Experience

- By providing seamless transitions and interactions, SPAs mimic desktop applications, creating a more fluid user experience.

Client-Side Rendering

- Most rendering occurs on the client side, reducing the load on the server and enabling faster interactions after the initial load.

API Integration

- SPAs typically communicate with back-end services through APIs, allowing for efficient data retrieval and manipulation.

Reduced Server Load

- By offloading more processing to the client side, SPAs decrease the demand on server resources.

Modular Development

- SPAs encourage modular architecture, making it easier to maintain and develop by dividing the application into reusable components.

Do not neglect these #1

Client-Server Pattern

- Single point of failure – If the server goes down
- Scalability challenges – with increasing number of clients
- Latency – possible network delays
- Security risks – server is the prime target for attacks

Microservices

- Distributed – complexity in deployment and orchestration
- Inter-service communication – overhead in calls
- Data consistency – Including separated databases
- Debugging difficulty – complex Troubleshooting
- Coordination – cumbersome and error-prone

Do not neglect these #2

Progressive Web Apps

- Inconsistent support – features may not be fully across all browsers
- Performance variability – affected by network conditions
- Storage limitations – Browsers impose limits on local data
- Complex management – delivery and management of service workers

Single-Page Applications

- Load time – longer initial time of loading
- Compatibility – rely heavily on JavaScript features
- Memory issues – excessive memory consumption in the browser
- State management – the application's state can become complicated

CORE TECHNOLOGIES

HTML

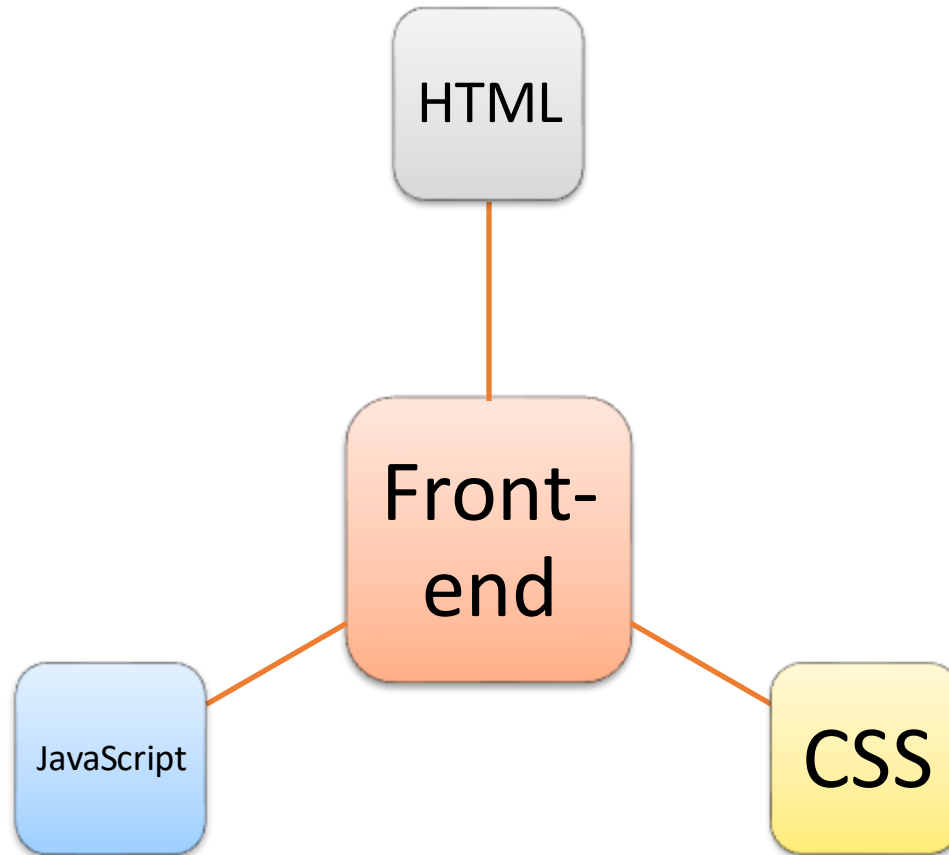
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>A test page</title>
  </head>
  <body>
    
  </body>
</html>
```

CSS

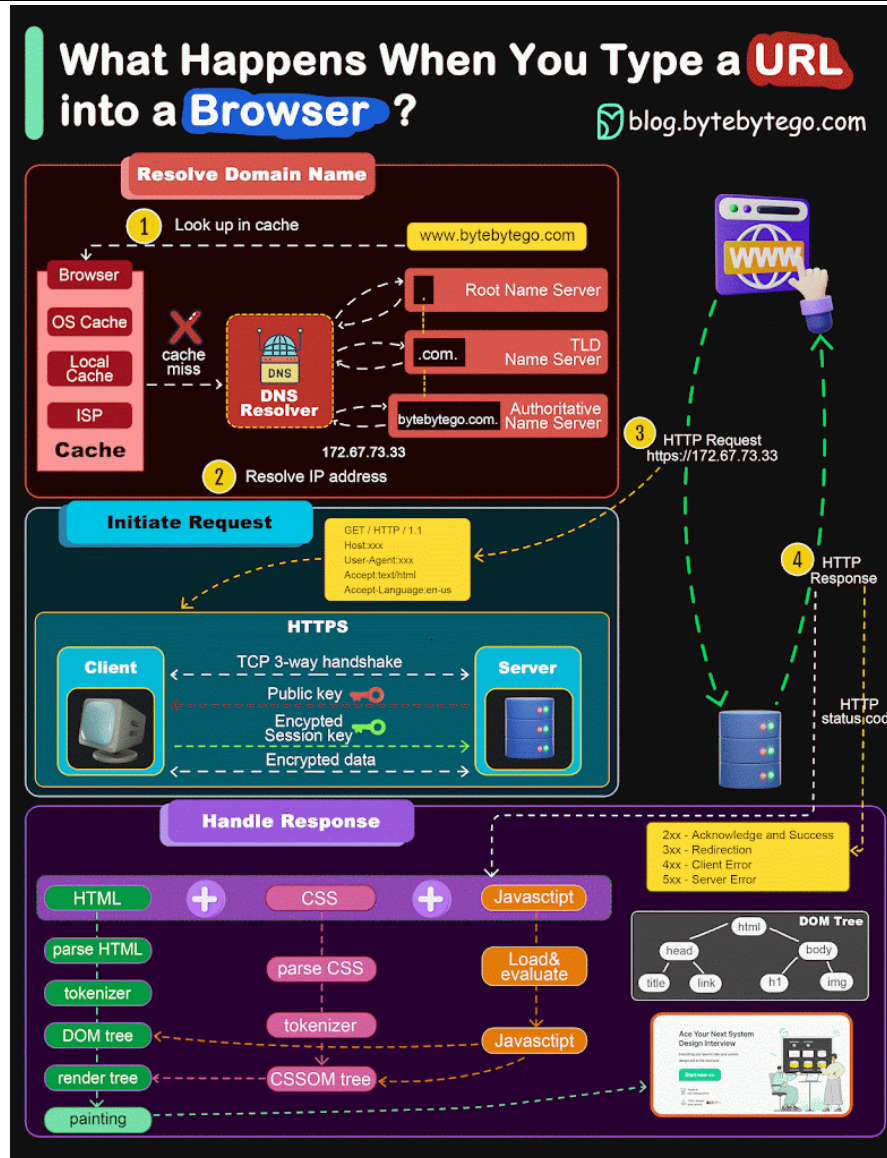
```
article, aside, details, figcaption, figure, footer, header,
hgroup, main, menu, nav, section, summary {
    display: block
}
audio, canvas, progress, video {
    display: inline-block;
    vertical-align: baseline
}
audio:not([controls]) {
    display: none;
    height: 0
}
```

A partial example from ZUT.edu.pl

The triangle of web technologies

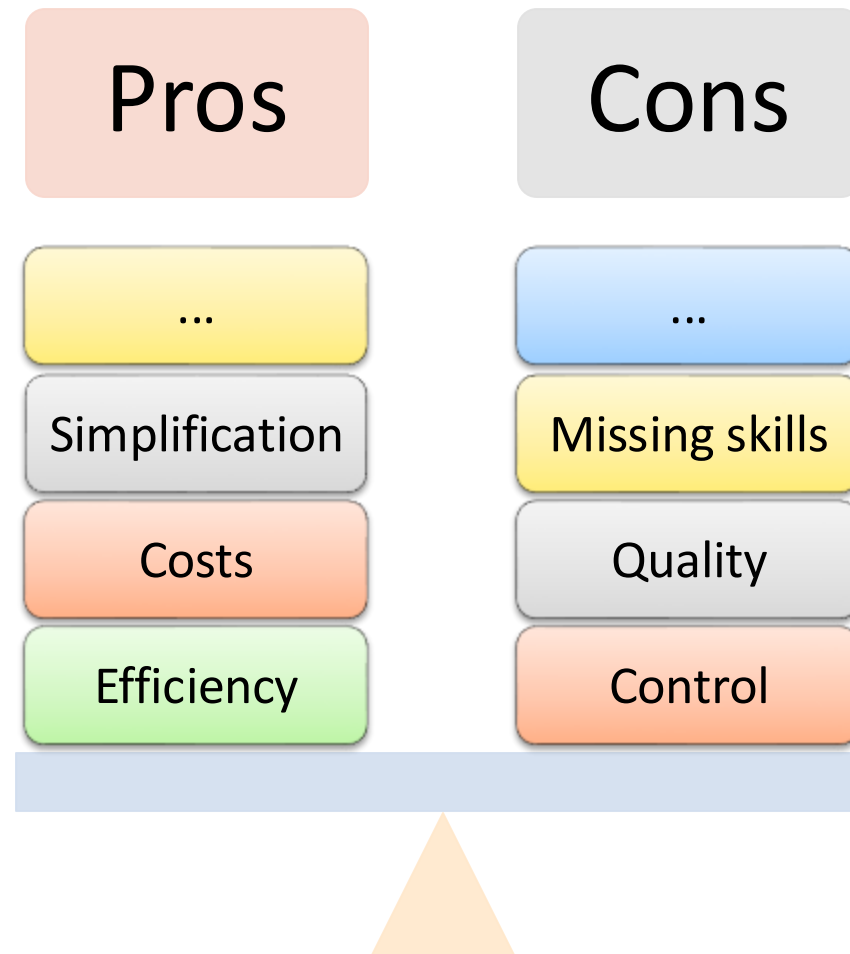


Dynamic Documents and Front-end Web Development

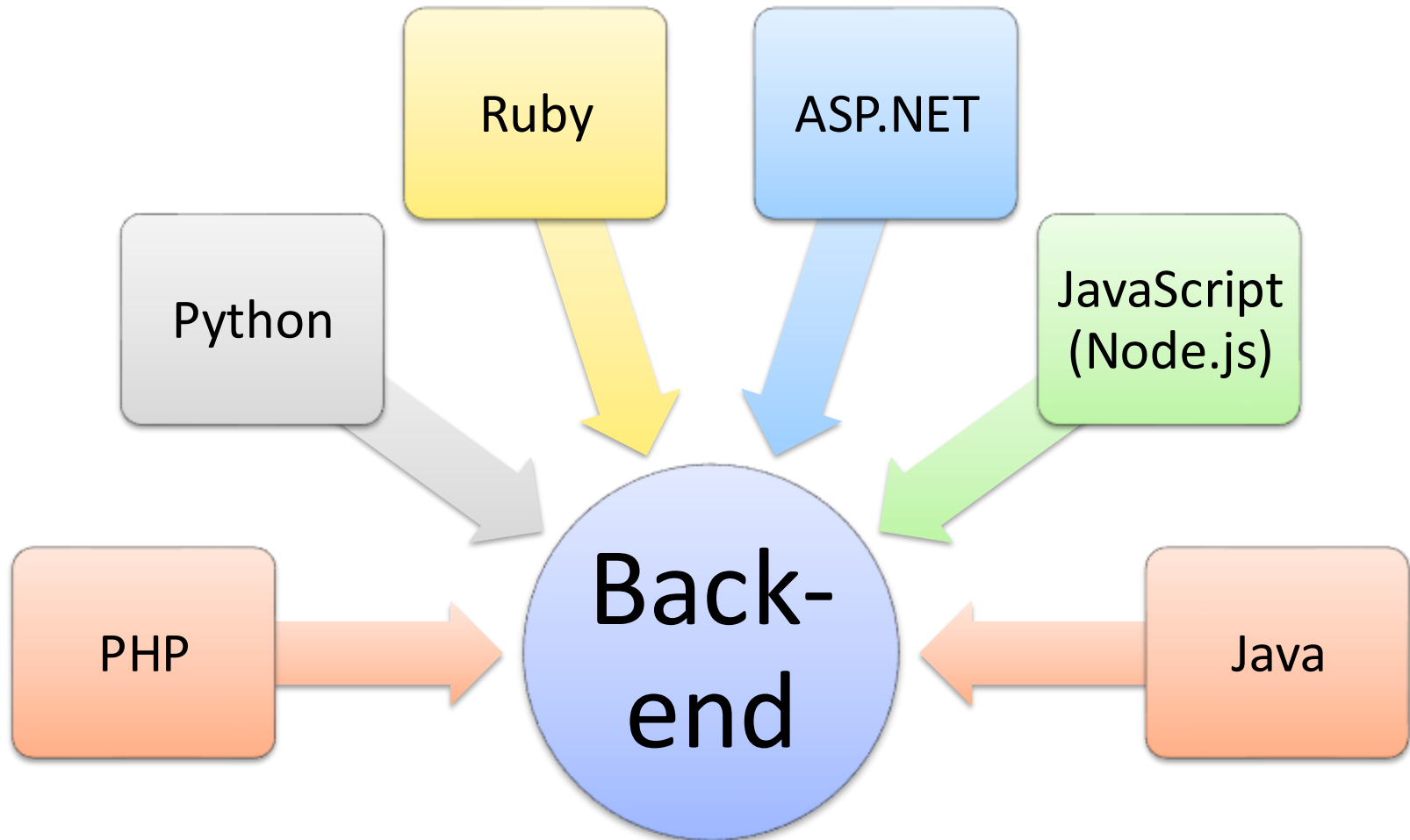


Note: Check
ByteByteGo
YouTube
channel

AI & Web Development



Back-end languages – to be discussed later



Summary

- Core technologies – JavaScript, HTML & CSS
- Standards supported by web browsers
- Stable situation for multiple years (it is an evolution, not a revolution anymore)
- Increasing compatibility (mature market)
- Front-end development is a part of Web apps
- Mobile websites are not inferior to “desktop” ones

TILL NEXT LECTURE